JOB PLANNER
LIVRABLE 1

Nom du projet	**Job Planner** (Centralisation de la recherche d'emploi)
Auteur	Sacha Duvivier
Cours	**Projet de Master** (Livrable 1)

Introduction

JobPlanner est une plateforme (application web et PWA*) qui aide les candidats à centraliser et piloter leur recherche d'emploi à l'instar de Indeed ou LinkedIn.

Aujourd'hui, il existe beaucoup de plateformes comme LinkedIn ou Indeed qui facilitent l'accès aux offres d'emploi, mais ces dernières proposent peu d'outils avancés pour le suivi personnalisé des candidatures. En effet, les outils ATS* sont principalement destinés aux recruteurs.

L'application vise à combler ce manque, et donc devenir une véritable boite à outils pour le candidat. Plus qu'un simple outil, cette plateforme prend en compte tout le processus de recherche d'emploi: le suivi des offres, des candidatures, des échanges avec les recruteurs, des relances et des entretiens.

Problématique et besoins

L'État de l'art:
Certaines plateformes comme JobCopilot, Huntr, Trello, Relance Work pourraient remplir ce besoin mais elles restent encore très peu utilisés, et payants pour la plupart, on innove avec ce

projet pour son approche automatisée et centralisée pour le candidat, À contrario de ces derniers, ce n'est pas juste un tracker passif pour postuler aux candidatures.

Contrairement à Huntr ou JobCopilot qui se limitent à l'autocomplétion, la visualisation, ou encore à l'entrainement fictif d'entretiens.

Ici, on va synchroniser activement les mails reçus par le candidat, les recruteurs pour ne rien rater, avec une synthèse des réponses, et notamment des entretiens.

Les outils comme RelanceWork scannent Gmail mais restent sur des templates fixes, sans historique des interactions ou génération de questions d'entretien à la volée, surtout qu'il est payant au bout de 10 candidatures et ne prend pas en compte le tracking d'email avec la version gratuite.

Huntr excelle mais reste une solution payante au bout de 100 candidatures, et ne prend pas en compte le scan de mail, et ce dernier est plus basé sur la génération de CV par IA et du kanban.

Trello/Notion demandent setup manuel sans synchronisation email.

JobPlanner a pour but d'être open-source, self-hosted et gratuit. Il a pour but d'être complètement gratuit, d'utiliser ollama ou pouvoir connecter sa propre api pour la génération IA.

Concrètement c'est la première vraie boite à outil professionnelle qui pilote du début jusqu'à la fin toutes les offres d'emploi, pas un simple tableur digital.

La recherche d'emploi implique l'utilisation de multiples outils: emails, excel, notes, plateformes externes ce qui rend le suivi assez complexe pour le candidat.

Les principaux besoins identifiés sont les suivants:

- Centraliser les informations liées aux candidatures (liens, différentes plateformes, recherche de mail du recruteur ou d'informations sur l'entreprise)
- Ne jamais rater un seul mail de la part d'un recruteur (opportunités manquées)
- Améliorer la réactivité et la qualité des échanges

On vient remplacer les suivis manuels (Excel et autres..) par l'automatisation des informations liées aux candidatures, automatiser les tâches répétitives (de relances, de suivi), ne plus manquer d'opportunités, générer des réponses pertinentes rapidement mais aussi, envoyer des mails de remerciements automatiques personnalisés.

Ces problématiques vont être répondu grâce à ce projet, Via l'extension web, on a la possibilité de récupérer une offre d'emploi en un clic, récupérer les informations pertinentes, comme le langage, les outils utilisés, l'historique de l'entreprise.

Planning prévisionnel

Le projet est organisé en phases de sprints, avec des livrables.

Date	Objet	Statut
Jusqu'au 31 mars	**Cadrage & L1 + MVP existant**	**Compréhension des besoins, Etat de l'art, objectifs, planning, organisation + démonstration MVP / POC / API**
Début avril → 28 avril	Consolidation & conception	architecture stabilisée, modèles de données alignés avec l'existant, ajout d'améliorations.
Avril → mi-mai	Développement avancé	Améliorations complétées, intégration frontend/backend, API finalisée, remplacement des mocks
Mi-mai → 21 mai	Stabilisation & livraison	Tests, corrections, version complète exploitable, rapport final

1 - Objectifs fonctionnels

Les objectifs fonctionnels de la platforme sont les suivants:

- Afficher le suivi des offres et des candidatures dans un workflow clair, avec une vue kanban pour une gestion facile comme sur Notion.
- Pouvoir facilement ajouter une offre d'emploi dans le système sans devoir tout écrire à la main.
- Historiser les interactions (emails, actions, changements de statut).
- Faciliter la planification des entretiens et des actions suivantes.
- Automatiser les tâches répétitives (synchronisation, relances, rappels).

Mais surtout, dans le but d'améliorer la productivité utilisateur via l'IA (extraction, résumé, génération de mails).

2 - Mise en application

Dans le contexte du projet de master, certains points seront mis en évidence tout au long du projet:

- Tout d’abord, la mise en pratique de la gestion de projet Agile (planning, priorisation, suivi) via un outil de ticketing comme Jira.
- Mobiliser les compétences techniques apprises lors de la licence et du master: (backend Symfony, frontend Vue, PostgresSQL, sécurité/JWT/hashing)
- Intégrer les notions de qualite logicielle (validation, tests, documentation, versionning).

3- Méthode de travail

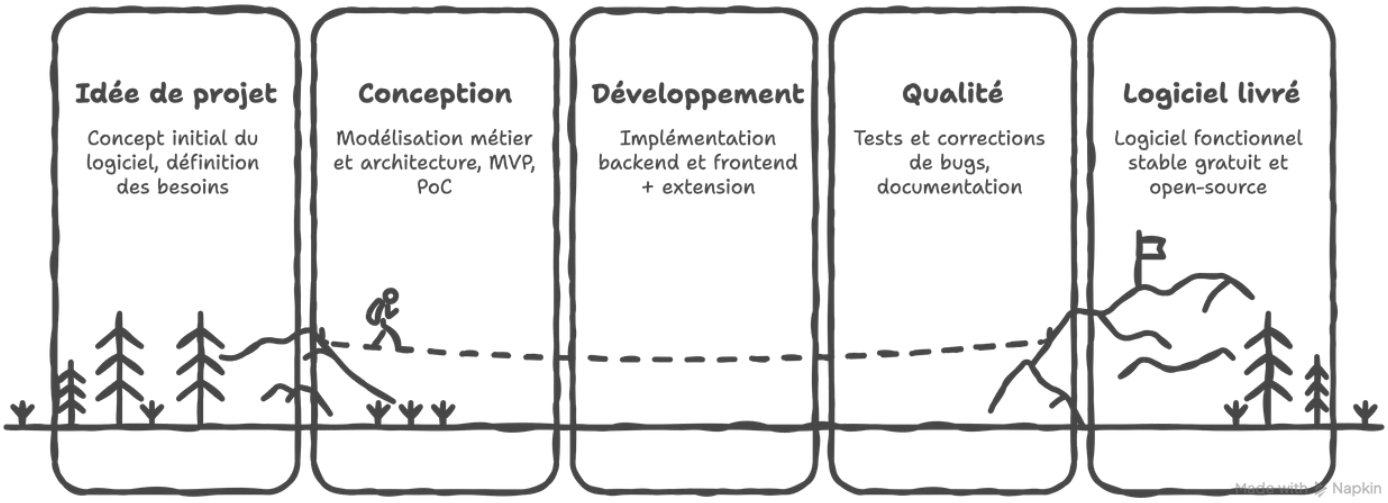
Concernant ma méthode de travail, elle est organisée assez simplement, en mini-sprints avec objectifs hebdomadaires.
→ Il y a notamment un suivi des tâches par priorités (PoC*, MVP*, Mock*) ainsi que des délais à respecter afin d’être en adéquation avec les délais de rendus.

Ensuite, chaque fin de sprint je prend du temps pour la mise au point de l'avancement chaque partie de développement et surtout de la roadmap pour pouvoir ajuster via le planning.

Dès qu’un ticket est cloturé, une documentation est remplie sur ce dernier, pour expliquer les decisions techniques et fonctionnelles mais aussi les ajouts, et difficultés rencontrées.

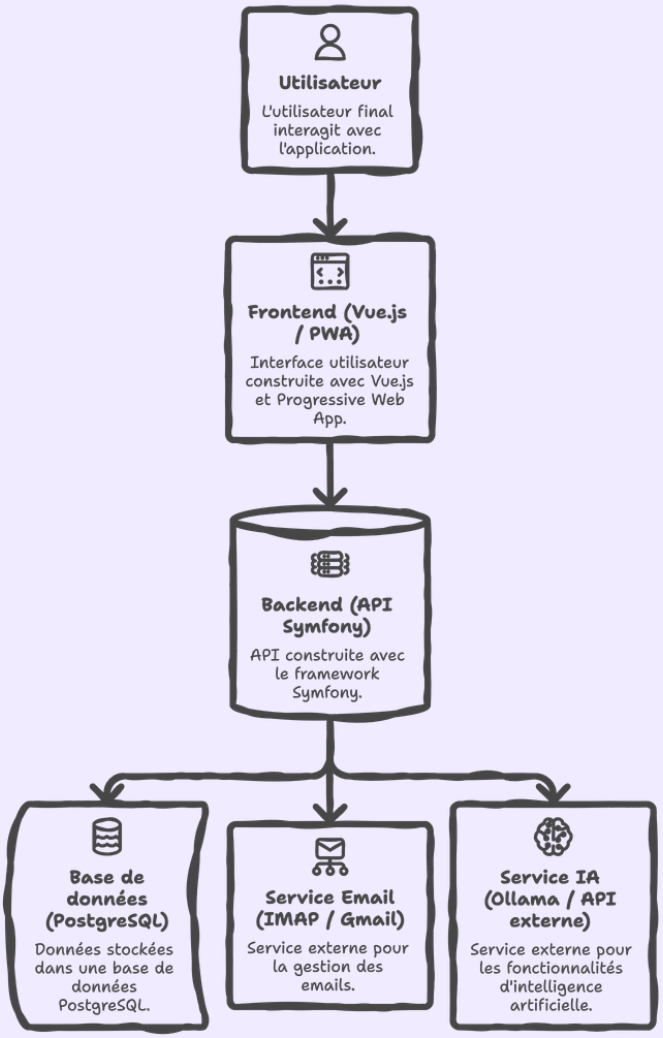
4- Répartition du projet

Afin de garder une cohérence avec le projet, j’ai divisé le projet en 5 grandes étapes mais qui sont utilisé à chaque itération de développement



Étant seul pour le projet, tout sera développé par mes soins mais en gardant une démarche collaborative pour la volonté de le rendre open-source.

5- Architecture



Architecture

Coté architecture, j'ai décidé de faire une application assez simple fonctionnellement, mais puissante.

Un frontend en PWA avec VueJS, un backend Symfony qui utilise une base de donnée Postgres et l'utilisations de services externes pour ce qui concerne l'IA.

6- Risques identifiés

J'ai identifiés quelques risques au départ, notamment vis à vis de l'intégration des mails (assez complexe pour l'utilisateur et coté sécurité/authentification).

Mais aussi coté association automatiques des mails car peut être pas assez précis si plusieurs interlocuteurs.

Le coût de l'intelligence artificielle pour le candidat ou des limitations techniques en fonction de l'API choisie par ce dernier.

Et enfin coté gestion des données sensibles pour garder tout cela RGPD.

Annexe:

Extrait des User Story* restantes:

- Choisir la stratégie d'intégration email (pour l'instant IMAP)
- Définir les règles de liaison entre emails et candidatures (via une regex ou par IA)
- Définir l'intégration de l'IA (usage, coût pour l'utilisateur)
- Définir le périmètre de l'extension navigateur → Ajout automatique via une connexion et un Bearer ou un lien.
- Gérer les aspects sécurité (emails, tokens, données sensibles)
- Améliorer l'implémentation de l'API
- Amélioration UX/UI de l'application
- Gestion multi-comptes ou unique

Un exemple:



User Story: SU-12

En tant que candidat, je veux ajouter une offre d'emploi directement depuis mon navigateur afin de gagner du temps lors de mes recherches.

Critères d'acceptation

- Une extension permet de récupérer les données d'une offre
- Une candidature est créée automatiquement
- L'utilisateur est authentifié via token ou session sécurisée

Dictionnaire technique du premier livrable:

ATS: Applicant Tracking System → logiciel utilisé pour gérer les candidatures à un poste, recevoir les candidatures mais surtout suivre chaque candidat dans le processus de recrutement.

PWA: Progressive Web App → Application Web se comportant comme une application mobile native.

Mock → Version simulée des fonctionnalités

PoC: Proof of Concept → Prototype et démonstration technique pour valider la faisabilité de la solution.

User Story: description d'un besoin utilisateur sous forme de carte